COS120 - CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BÀI 4: DANH SÁCH

MỤC TIÊU

- Hiểu về cấu trúc dữ liệu dạng **danh sách**, cách tổ chức lưu trữ và các **thao tác cơ bản** trên danh sách.
- Phân biệt và lựa chọn sử dụng danh sách liền kề (array list) và danh dách **liên kết** (linked list).
- Nắm được các phương pháp và thuật toán để hiện thực các thao tác cơ bản trên hai **loại danh sách**.

NỘI DUNG

- 4.1 - TỔNG QUAN VỀ DANH SÁCH
- 4.2 - DANH SÁCH **LIỀN KỀ**
- 4.3 - DANH SÁCH **LIÊN KẾT**
- 4.4 - HIỆN THỰC DANH SÁCH LIÊN KẾT ĐƠN

DANH SÁCH

4.1 - TỔNG QUAN VỀ DANH SÁCH

KHÁI NIỆM

- Danh sách (list) là một tập hợp các phần tử được tổ chức theo một **thứ tự nhất định**.
- Danh sách là một trong những cấu trúc dữ liệu **quan trọng** và phổ biến.
- Các thao tác trên danh sách: thêm, xóa, sắp xếp, tìm kiếm, mở rộng,…
- Các loại danh sách: danh sách liền kề, danh sách liên kết đơn, danh sách liên **kết kép**, danh sách liên kết vòng,…

CÁC TÁC VỤ CỦA DANH SÁCH

CÁC TÁC VỤ CƠ BẢN

- Khởi động DS (Initialize)
- Nhập danh sách (List Input)
- Xuất danh sách (List Output)
- Truy cập phần tử (Access)
- Chèn phần tử (Insertion)
- Xóa phần tử (Deletion)
- Kiểm tra kích thước (Size Checking)
- Kiểm tra DS rỗng (Empty Checking)
- Thay đổi giá trị phần tử (Update)
- Tìm kiếm phần tử (Search)

CÁC TÁC VỤ CỦA DANH SÁCH

CÁC TÁC VỤ NÂNG CAO

- Sắp xếp danh sách (Sorting)
- Sao chép danh sách (Copying)
- Ghép nối DS (Concatenation)
- Cắt danh sách (Slicing)
- Mở rộng danh sách (Extend)
- Thu nhỏ danh sách (Shrink)

……

CÁC PHƯƠNG PHÁP CÀI ĐẶT

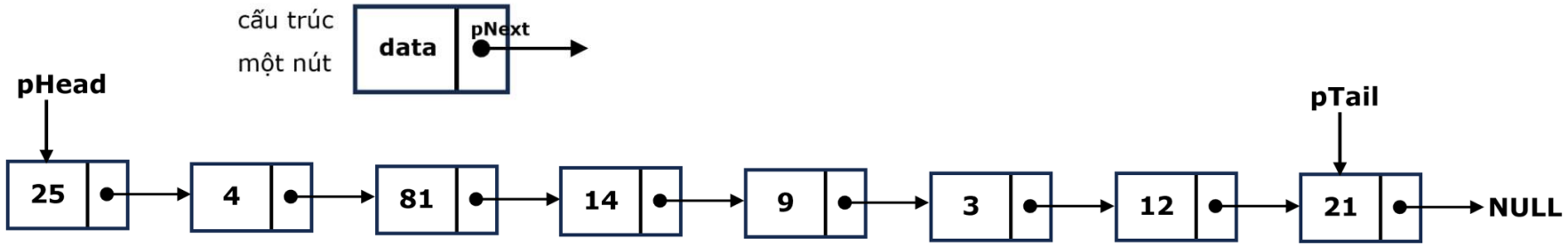
❑ Cài đặt bằng phương pháp **danh sách liền kề**

- các phần tử được bố trí kế cận nhau, liên tục nhau trong bộ nhớ máy tính.

25	4	81	14	9	3	12	21
0	1	2	3	4	5	6	7

❑ Cài đặt bằng phương pháp **danh sách liên kết**

- một danh sách bao gồm các nút, **mỗi nút** có trường **data** và trường **pNext** chỉ đến nút kế tiếp.



DANH SÁCH

4.2 - DANH SÁCH LIỀN KỀ

KHAI BÁO CẤU TRÚC DANH SÁCH LIỀN KỀ

Một danh sách liền kề gồm hai trường:

- *Trường **num**:* lưu số lượng nút hiện có trong danh sách.
- *Trường **nodes***: là **mảng một chiều**, mỗi phần tử của mảng lưu trữ một dữ liệu của DS.

```
#define MAX 100

typedef struct tList {
    int num;
    int nodes[MAX];
}LIST;
```

CÀI ĐẶT CÁC TÁC VỤ CƠ BẢN

KHỞI ĐỘNG DANH SÁCH

- Đầu vào: một danh sách liền kề plist.
- Kết quả: danh sách plist có số lượng nút bằng 0.

```
1  void Init(LIST &plist)
2  {
3      plist.num = 0;
4  }
```

NHẬP DANH SÁCH

```
1  void InputList(LIST &plist)
2  {        int i;
3           do{
4                printf("Nhap so luong nut: ");
5                scanf("%d", &plist.num);
6           }while(plist.num<=0 || plist.num>MAX);
7
8           for (i=0; i<plist.num; i++)
9           {
10               printf("Nhap nut thu %d: ", i);
11               scanf("%d",&plist.nodes[i]);
12          }
13 }
```

CÀI ĐẶT CÁC TÁC VỤ CƠ BẢN

KIỂM TRA DANH SÁCH RỖNG

```
int IsEmpty(LIST plist)
{
        return (plist.num == 0); //1-true  0-false
}
```

KIỂM TRA DANH SÁCH ĐẦY

```
int IsFull(LIST plist)
{
        return (plist.num == MAX);
}
```

CÀI ĐẶT CÁC TÁC VỤ CƠ BẢN

TRUY XUẤT MỘT PHẦN TỬ CỦA DANH SÁCH

```
int Retrieve(LIST plist, int pos, int &x)
{         if (IsEmpty(plist))
          {    printf("Danh sach bi rong.");
               return 0; //khong thanh cong
          }

          if (pos<0 || pos>=ListSize(plist))
          {
               printf("Vi tri %d khong hop le.", pos);
               return 0; //khong thanh cong
          }

          x=plist.nodes[pos];
          return 1; //truy xuat thanh cong
}
```

CÀI ĐẶT CÁC TÁC VỤ CƠ BẢN

CHÈN MỘT PHẦN TỬ MỚI VÀO DANH SÁCH

```
int Insert(LIST &plist, int pos, int x)
{         int i;
          if (IsFull(plist))
          {    printf("Danh Sach bi day.");
               return 0; //khong thanh cong
          }

          if (pos<0 || pos>ListSize(plist))
          {    printf("Vi tri %d khong hop le.", pos);
               return 0; //khong thanh cong
          }

          for (i=ListSize(plist)-1; i>=pos; i--)
               plist.nodes[i+1] = plist.nodes[i];
          plist.nodes[pos] = x;
          plist.num++;
          return 1; // chen thanh cong
}
```

CÀI ĐẶT CÁC TÁC VỤ CƠ BẢN

XÓA MỘT PHẦN TỬ RA KHỎI DANH SÁCH

```
int Remove(LIST &plist, int pos, int &x)
{
        int i;
        if (IsEmpty(plist))
        {
            printf("Danh sach rong.");
            return 0; //khong thanh cong
        }

        if (pos<0 || pos>=ListSize(plist))
        {
            printf("Vi tri %d khong hop le.", pos);
            return 0; //khong thanh cong
        }

        x = plist.nodes[pos];
        for (i=pos; i<ListSize(plist)-1; i++)
            plist.nodes[i] = plist.nodes[i + 1];
        plist.num--;
        return 1; // xoa thanh cong
}
```

CÀI ĐẶT CÁC TÁC VỤ CƠ BẢN

TÌM KIẾM MỘT PHẦN TỬ TRONG DANH SÁCH

```
int Search(LIST plist, int x)
{           int i;
            for(i=0; i<plist.num; i++)
                if(plist.nodes[i]==x)
                    return i;
            return -1;
}
```

Xem thêm một số thao tác khác trong giáo trình

DUYỆT DANH SÁCH

```
void ShowList(LIST plist)
{           int i;
            if(plist.num == 0){
                printf("Danh sach rong.");
                return;
            }
            for (i=0; i<plist.num; i++)
                printf("%d ", plist.nodes[i]);
}
```

Bài tập Cài đặt bằng phương pháp danh sách liền kề

BÀI 1: Dùng danh sách liền kề để hiện thực các tác vụ sau đây:

1) Định nghĩa danh sách các sinh viên, mỗi sinh viên gồm các thông tin: mã sinh viên char[10], tên sinh viên [40], điểm trung bình float.
2) Tính trung bình cộng điểm trung bình của sinh viên có điểm trung bình >7 trong danh sách
3) Tìm sinh viên có điểm trung bình lớn nhất trong danh sách
4) Sắp xếp danh sách sinh viên tăng dần theo điểm trung bình
5) Sắp xếp danh sách sinh viên tăng dần theo tên sinh viên

DANH SÁCH

4.3 - DANH SÁCH LIÊN KẾT

CÁC LOẠI DANH SÁCH LIÊN KẾT

- **Danh sách liên kết đơn (Singly linked list):** Mỗi nút chỉ chứa một con trỏ **trỏ** tới nút tiếp theo trong danh sách.

- **Danh sách liên kết đôi (Doubly linked list):** Mỗi nút chứa **hai con trỏ**, một trỏ tới nút tiếp theo và một trỏ tới nút trước đó trong danh sách.

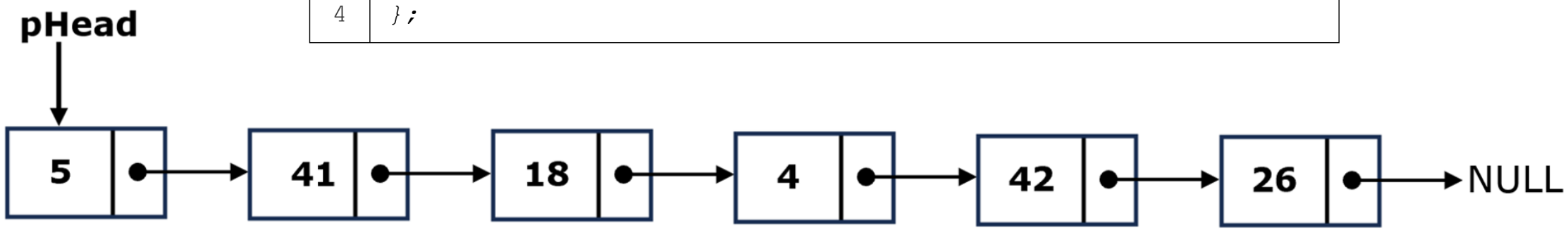
- **Danh sách liên kết vòng (Circular linked list):** Trong danh sách này, **nút cuối** cùng trỏ tới nút **đầu tiên**, tạo thành một vòng tròn.

DANH SÁCH LIÊN KẾT ĐƠN

DSLK đơn là một chuỗi các nút (node) được liên kết với nhau. Một nút gồm hai thành phần:

- Dữ liệu (**Data**): là thông tin cần lưu trữ cho mỗi phần tử trong danh sách.
- Con trỏ (**Pointer**): là một con trỏ trỏ tới nút tiếp theo trong danh sách.

```
struct Node {
    Data data;          // Dữ liệu của nút
    struct Node* pNext;  // Con trỏ trỏ tới nút tiếp theo
};
```

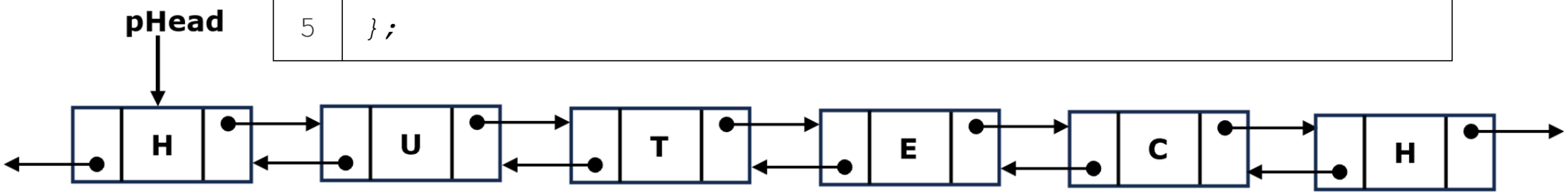


DANH SÁCH LIÊN KẾT ĐÔI

Một DSLK đôi là một chuỗi các nút (nodes) được liên kết với nhau. Mỗi nút có khả năng trỏ tới cả nút trước đó và nút tiếp theo trong danh sách. Một nút bao gồm ba thành phần:

- Dữ liệu (**Data**)
- Con trỏ đến nút tiếp theo (Next Pointer)
- Con trỏ đến nút trước đó (**Previous Pointer**)

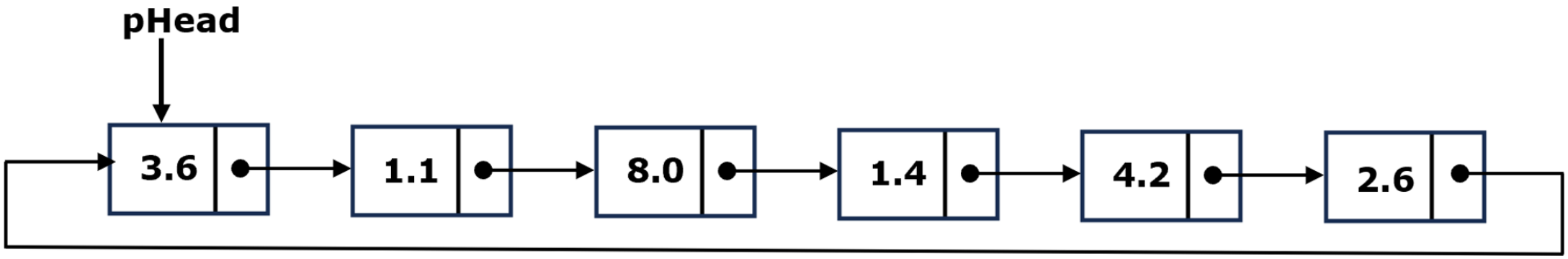
```
struct Node {
    Data data;                  // Dữ liệu của nút
    struct Node* pNext;         // Con trỏ trỏ tới nút tiếp theo
    struct Node* pPrevious;     // Con trỏ trỏ tới nút trước đó
};
```



DANH SÁCH LIÊN KẾT VÒNG

- Tương tự như DSLK đơn, **nhưng** con trỏ pNext của nút cuối sẽ trỏ tới nút đầu tiên của danh sách.

```
struct Node {
    Data data;                  // Dữ liệu của nút
    struct Node* pNext;          // Con trỏ trỏ tới nút tiếp theo
};
```

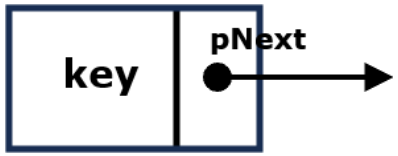


DANH SÁCH

4.4 - HIỆN THỰC DANH SÁCH LIÊN KẾT ĐƠN

XÂY DỰNG CẤU TRÚC NÚT VÀ DANH SÁCH

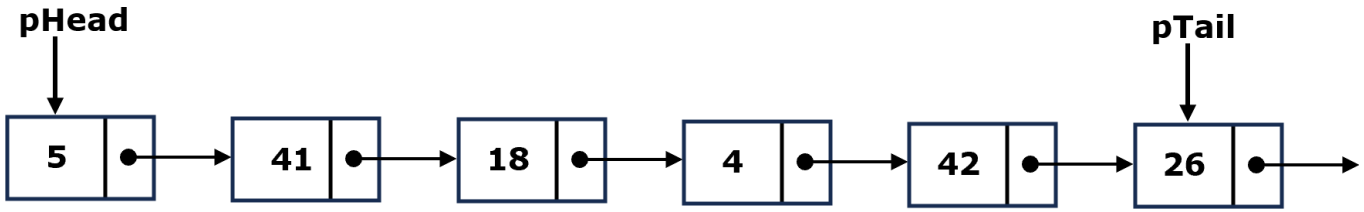
CẤU TRÚC MỘT NÚT




```
1  struct Node {
2      int key;              // ta gọi dữ liệu int của nút là key
3      struct Node* pNext;   // con trỏ trỏ tới nút tiếp theo
4  };
5
6  typedef struct Node NODE;  //đặt lại kiểu struct này là NODE
```

CẤU TRÚC MỘT DSLK ĐƠN

```
1  struct List {
2      NODE *pHead;  // Con trỏ quản lý nút đầu danh sách
3      NODE *pTail;  // Con trỏ quản lý nút cuối danh sách
4  };
5
6  typedef struct List LIST;  //đặt lại tên struct này là LIST
```



XÂY DỰNG CẤU TRÚC NÚT VÀ DANH SÁCH

KHỞI ĐỘNG DANH SÁCH

```
1  void Init(LIST &l)
2  {
3      l.pHead = NULL;
4      l.pTail = NULL;
5  }
6  // hoặc viết gọn: l.pHead = l.pTail = NULL;
```

KIỂM TRA DANH SÁCH RỖNG

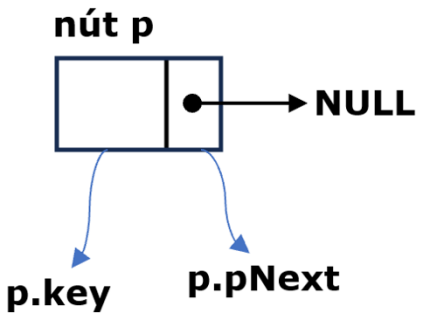
```
1  int IsEmpty(LIST l)
2  {
3      if(l.pHead == NULL)
4          return 1;
5      return 0;
6  }
7  // hoặc viết gọn: return (l.pHead == NULL);
```

XÂY DỰNG CẤU TRÚC NÚT VÀ DANH SÁCH

TẠO MỘT NÚT

```
NODE* CreateNode(int x)
{
    NODE* p = (NODE*)malloc(sizeof(NODE));
    if (p == NULL) {
        printf("Hết bộ nhớ!");
        exit(0);
    }
    p->key = x;
    p->pNext = NULL;
    return p;
}
// nếu dùng C++ thì cấp phát p = new NODE;
```

NODE p;



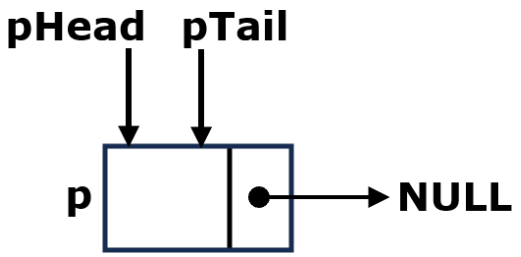
CÁC TÁC VỤ TRÊN DANH SÁCH LIÊN KẾT ĐƠN

- CÁC TÁC VỤ CHÈN
- CÁC TÁC VỤ NHẬP XUẤT DANH SÁCH
- TƯƠNG QUAN GIỮA DSLK ĐƠN VÀ MẢNG MỘT CHIỀU
- CÁC TÁC VỤ XÓA
- CÁC TÁC VỤ SẮP XẾP VÀ TÌM KIẾM
- MỘT SỐ TÁC VỤ KHÁC

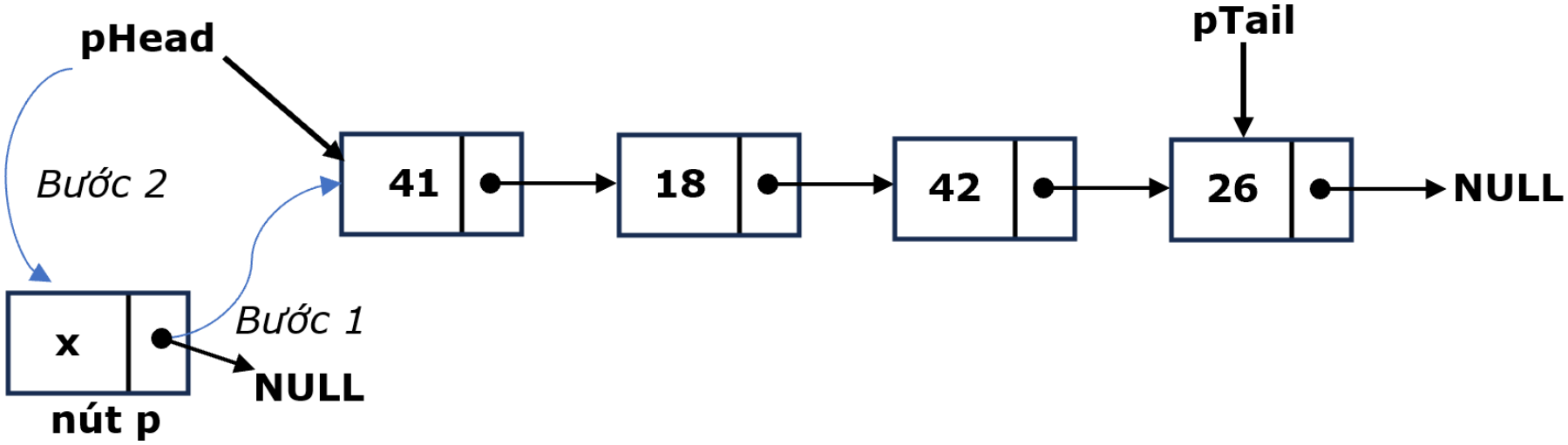
CHÈN MỘT NÚT VÀO ĐẦU DANH SÁCH

Gọi nút p là nút cần chèn

- **Trường hợp 1:** danh sách rỗng



- **Trường hợp 2:** danh sách không rỗng



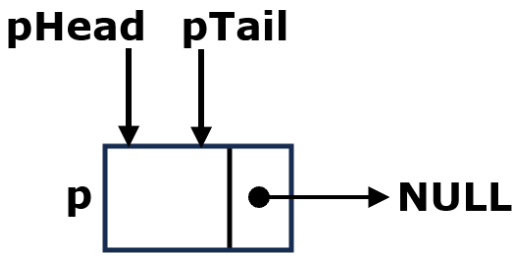
CHÈN MỘT NÚT VÀO ĐẦU DANH SÁCH

```
void InsertHead(LIST &l, NODE* p)
{
    if(l.pHead == NULL) //trường hợp danh sách rỗng
        l.pHead = l.pTail = p;
    else{ //truong hop danh sách không rỗng
        p->pNext = l.pHead; //Bước 1
        l.pHead = p;        //Bước 2
    }
}
```

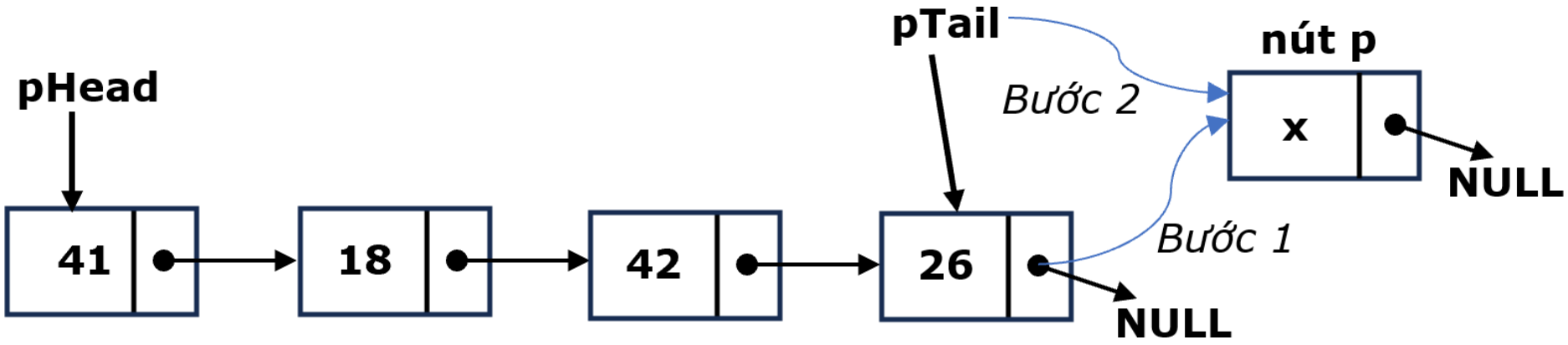
CHÈN MỘT NÚT VÀO CUỐI DANH SÁCH

Gọi nút p là nút cần chèn

- **Trường hợp 1:** danh sách rỗng



- **Trường hợp 2:** danh sách không rỗng



CHÈN MỘT NÚT VÀO CUỐI DANH SÁCH

```
1  void InsertTail(LIST &l, NODE* p)
2  {
3      if(l.pHead == NULL) //trường hợp danh sách rỗng
4          l.pHead = l.pTail = p;
5      else{ //truong hop danh sách không rỗng
6          l.pTail->pNext = p; //Bước 1
7          l.pTail = p;        //Bước 2
8      }
9  }
```

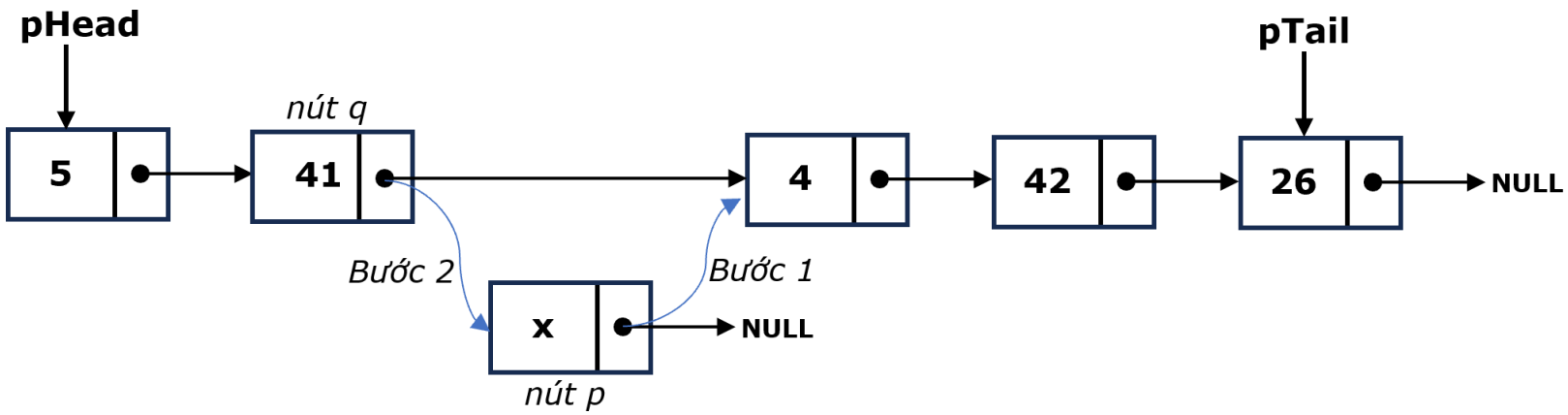
CHÈN MỘT NÚT VÀO VỊ TRÍ BẤT KỲ TRONG DS

Cho trước một nút q trong danh sách và một nút p **chưa** thuộc danh sách. Ta tiến hành chèn nút p vào danh sách **ngay sau** nút **q**. Giải thuật như sau:

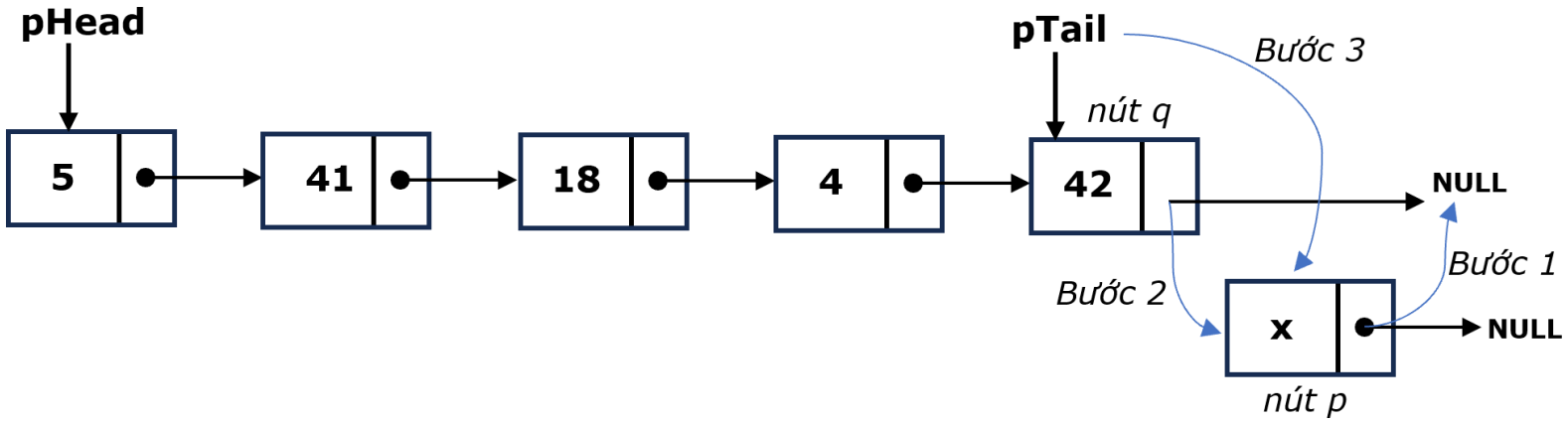
- **Trường hợp 1**: Nếu nút **q không tồn tại** thì ta không làm gì cả.
- **Trường hợp 2**: Ngược lại, nếu nút **q tồn tại**.
  - Bước 1: thực hiện p->pNext trỏ đến q->pNext.
  - Bước 2:
    - Thực hiện q->pNext trỏ đến p.
    - Bước 3: Nếu **q là nút pTail** thì cập nhật lại pTail mới là p. //nếu q là nút cuối DS

CHÈN MỘT NÚT VÀO VỊ TRÍ BẤT KỲ TRONG DS

- Hình vẽ mô tả các bước:



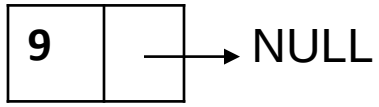
- Trường hợp nút q là nút pTail thì thực hiện thêm bước 3:



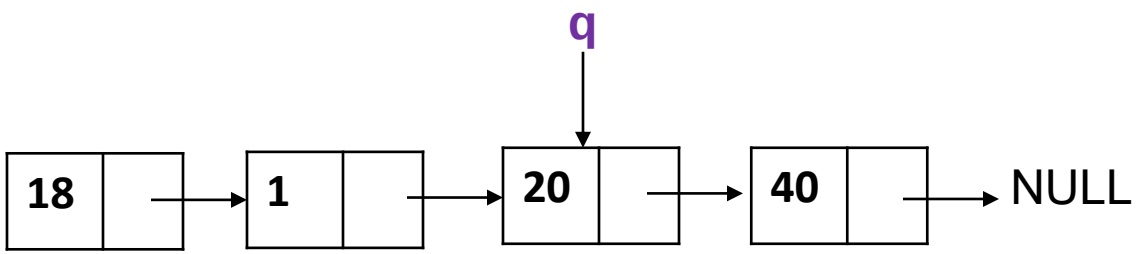
CHÈN MỘT NÚT VÀO VỊ TRÍ BẤT KỲ TRONG DS

```
void InsertAfter(LIST &l, NODE* q, NODE* p)
{
    if(q == NULL) return; //trường hợp 1
    //trường hợp 2
    p->pNext = q->pNext; //bước 1
    q->pNext = p; //bước 2
    if(q==l.pTail) //bước 3
        l.pTail = p;
}
```

BÀI TẬP

- Vẽ hình mô tả thực hiện chèn node p = 

1. Vào đầu danh sách
2. Vào cuối danh sách
3. Vào sau node q trong danh sách



NHẬP DANH SÁCH

- Khi nhập dữ liệu vào một danh sách liên kết đơn, ta phải thực hiện các bước sau:
  - Bước 1: Khởi tạo một danh sách rỗng.
  - Bước 2: Nhập dữ liệu x cho một nút.
  - Bước 3: Tạo nút mới p có dữ liệu là x.
  - Bước 4: Thêm nút p vào đầu danh sách.
  - Lặp lại bước 2.

  Chú ý, ở bước 4 ta có thể thêm p vào cuối danh sách.
- Để kết thúc việc nhập danh sách, ta có hai cách:
  - **Cách 1:** Ngay từ đầu, ta phải nhập số lượng nút cho danh sách là n.
  - **Cách 2:** Ta sẽ nhập cho đến khi muốn dừng thì ta cố tình nhập x bằng 0 (hoặc một giá trị x nào đó tùy ý mình).

NHẬP DANH SÁCH CÁCH 1

- Hàm nhập danh sách liên kết đơn theo cách 1:

```
void InputList(LIST &l)
{     int i,n,x;  NODE* p;
      Init(l);
      printf("Nhap so luong nut: ");
      scanf("%d",&n);
      for(i=0;i<n;i++)
      {     printf("Nhap data cua nut thu %d: ",i);
            scanf("%d",&x);
            p=CreateNode(x);
            InsertHead(l,p); //hoặc InsertTail(l,p)
      }
}
```

NHẬP DANH SÁCH CÁCH 2

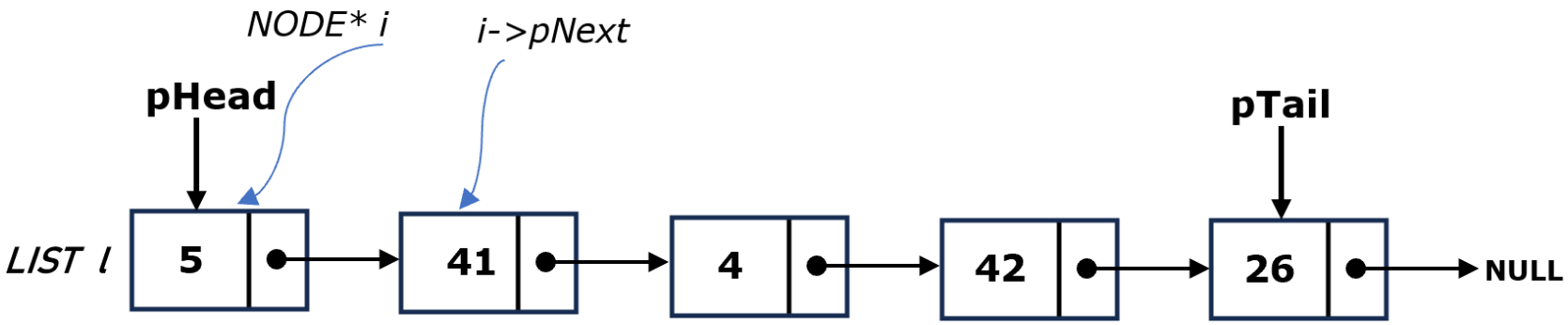
- Hàm nhập danh sách liên kết đơn theo cách 2:

```
void InputList(LIST &l)
{
    int i,x;
    NODE* p;
    Init(l);
    while(1)
    {
        printf("Nhap data cho nut: ");
        scanf("%d",&x);
        if(x==0) break;
        p=CreateNode(x);
        InsertHead(l,p); //hoặc InsertTail(l,p)
    }
}
```

DUYỆT DANH SÁCH

- Để duyệt một danh sách liên kết đơn, ta thực hiện các bước sau:
  - Bước 1: tạo một con trỏ **NODE* i**, cho **i trỏ đến nút pHead** của danh sách.
  - Bước 2: Xuất giá trị của nút mà **i** đang trỏ đến (xuất i->key).
  - Bước 3: di chuyển i sang nút kế tiếp (**i=i->pNext**).
  - Bước 4: Lặp lại bước 2.

Chú ý: ở bước 4, quá trình duyệt này phải dừng khi i trỏ đến NULL ngay sau nút pTail.



DUYỆT DANH SÁCH

Duyệt danh sách sử dụng vòng lặp WHILE

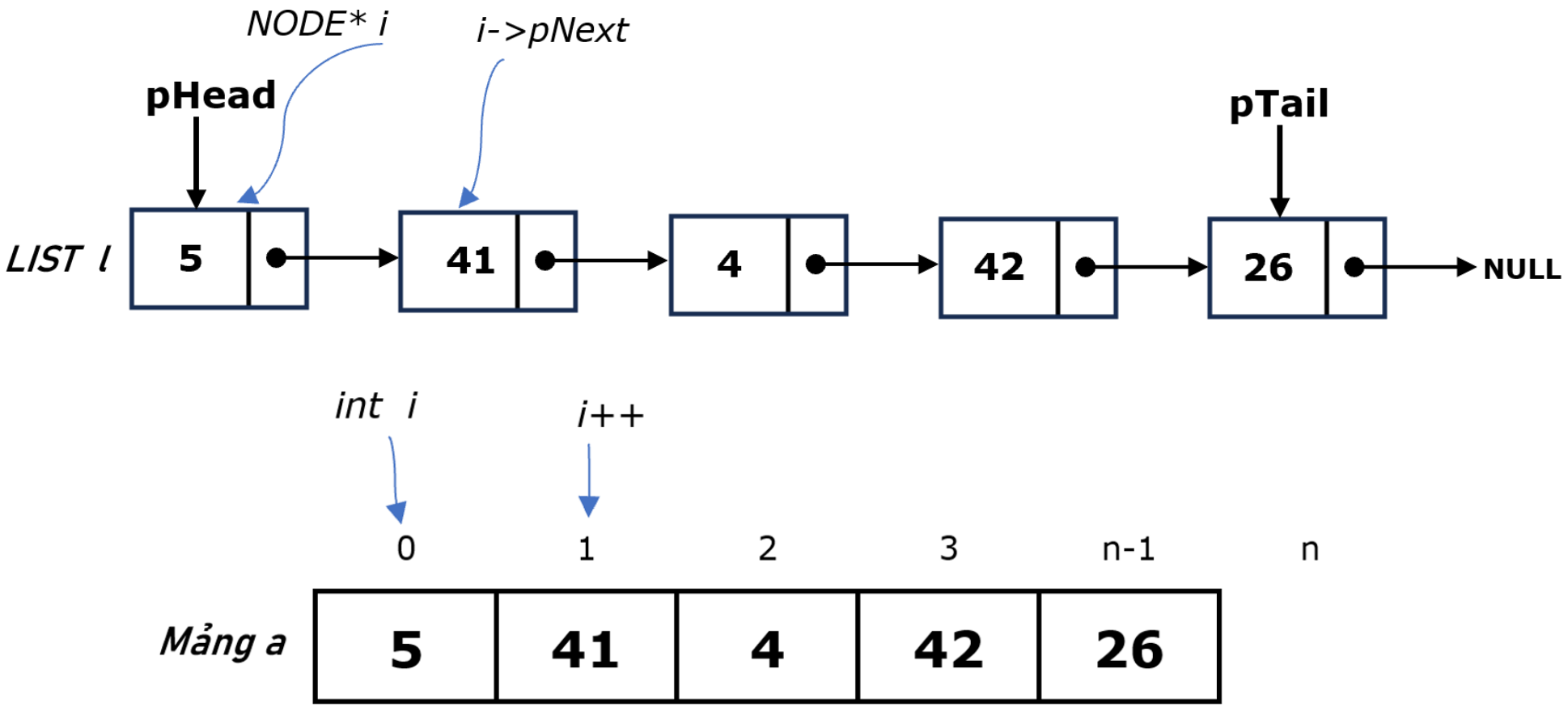
```
void OutputList(LIST l)
{     NODE *i;
      if(IsEmpty(l)==1)
      {
            printf("Danh sach rong");
            return;
      }

      i=l.pHead;
      while(i!=NULL)
      {     printf("%d->",i->key);
            i=i->pNext;
      }
}
```

DUYỆT DANH SÁCH

Duyệt danh sách sử dụng vòng lặp FOR

```
void OutputList(LIST l)
{
    NODE *i;
    if(IsEmpty(l)==1)
    {   printf("Danh sach rong");
        return;
    }
    for(i=l.pHead;i!=NULL;i=i->pNext)
        printf("%d->",i->key);
}
```

SỰ TƯƠNG QUAN GIỮA DSLK & MẢNG MỘT CHIỀU



SỰ TƯƠNG QUAN GIỮA DSLK & MẢNG MỘT CHIỀU

DANH SÁCH LIÊN KẾT ĐƠN	MẢNG MỘT CHIỀU

`NODE *i;`
`for(i=l.pHead;i!=NULL;i=i->pNext)`
`  printf("%d",i->key);`
	`int i;`
	`for(i=0;i<n;i++)`
	`  printf("%d ",a[i]);`

DANH SÁCH LIÊN KẾT ĐƠN	MẢNG MỘT CHIỀU
`NODE *i`	`int i`
`i=l.pHead`	`i=0`
`i=i->pNext`	`i++`
`i!=NULL`	`i<n`
`i!=l.pTail`	`i<n-1`
`i->key`	`a[i]`

SỰ TƯƠNG QUAN GIỮA DSLK & MẢNG MỘT CHIỀU

- **Áp dụng:** Chuyển đổi code tính tổng chẵn của mảng một chiều sang tổng chẵn của DSLK đơn

MẢNG MỘT CHIỀU	DANH SÁCH LIÊN KẾT ĐƠN

`int S=0;`
`int i;`
`for(i=0;i<n;i++)`
`  if(a[i] % 2 == 0)`
`  S += a[i];`
	`int S=0;`
	`NODE *i;`
	`for(i=l.pHead;i!=NULL;i=i->pNext)`
	`  if(i->key % 2 == 0)`
	`  S += i->key;`

BÀI TẬP TẠI danh sách liên kết đơn

Sử dụng danh sách liên kết đơn quản lý đa thức.

- **Mô tả: một đa thức trong toán học có dạng tổng quát:**

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$$

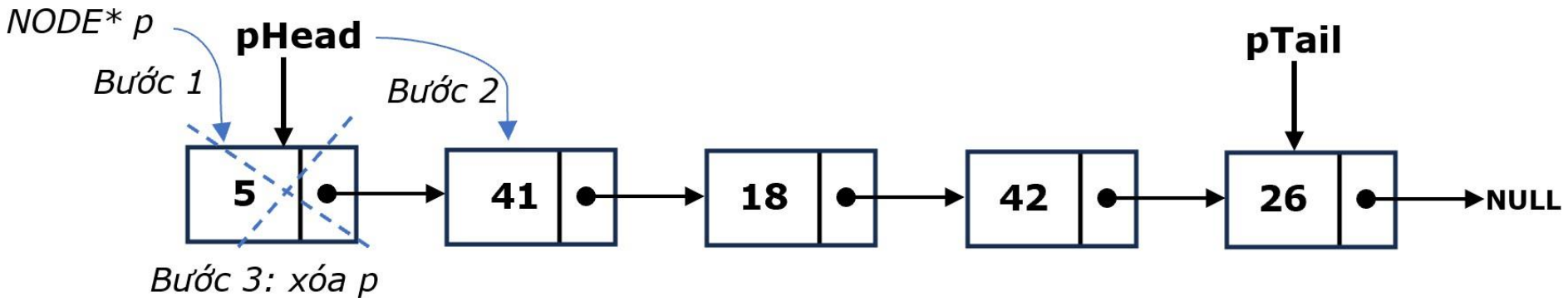
Các yêu cầu:

- **Khai báo cấu trúc dữ liệu: Sử dụng danh sách liên kết đơn để biểu diễn một đa thức. Mỗi nút (đơn thức) trong danh sách sẽ lưu trữ một hệ số và bậc của biến x.**
- **Viết hàm tạo một nút (đơn thức)**
- **Viết hàm chèn nút (đơn thức) vào đầu danh sách**
- **Viết hàm tìm số mũ cao trong đa thức**
- **Viết hàm tính giá trị của 1 node (1 đơn thức)**

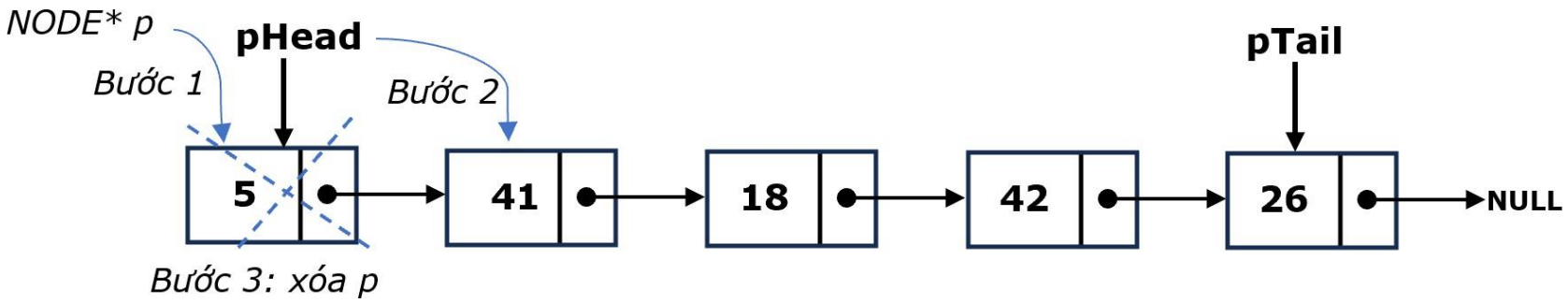
XÓA NÚT ĐẦU DANH SÁCH

CÁC BƯỚC THỰC HIỆN:

- Khai báo một con trỏ **p** kiểu NODE (NODE* p).
- **Nếu danh sách rỗng** thì ta không làm gì cả.
- **Nếu danh sách khác rỗng** thì ta thực hiện các bước sau:
  - Bước 1: cho p trỏ đến nút pHead.
  - Bước 2: di chuyển pHead đến nút ngay sau nút pHead.
  - Bước 3: hủy nút p.
  - Bước 4: nếu pHead mới bằng NULL thì danh sách đã bị rỗng, ta cũng cho pTail bằng NULL.



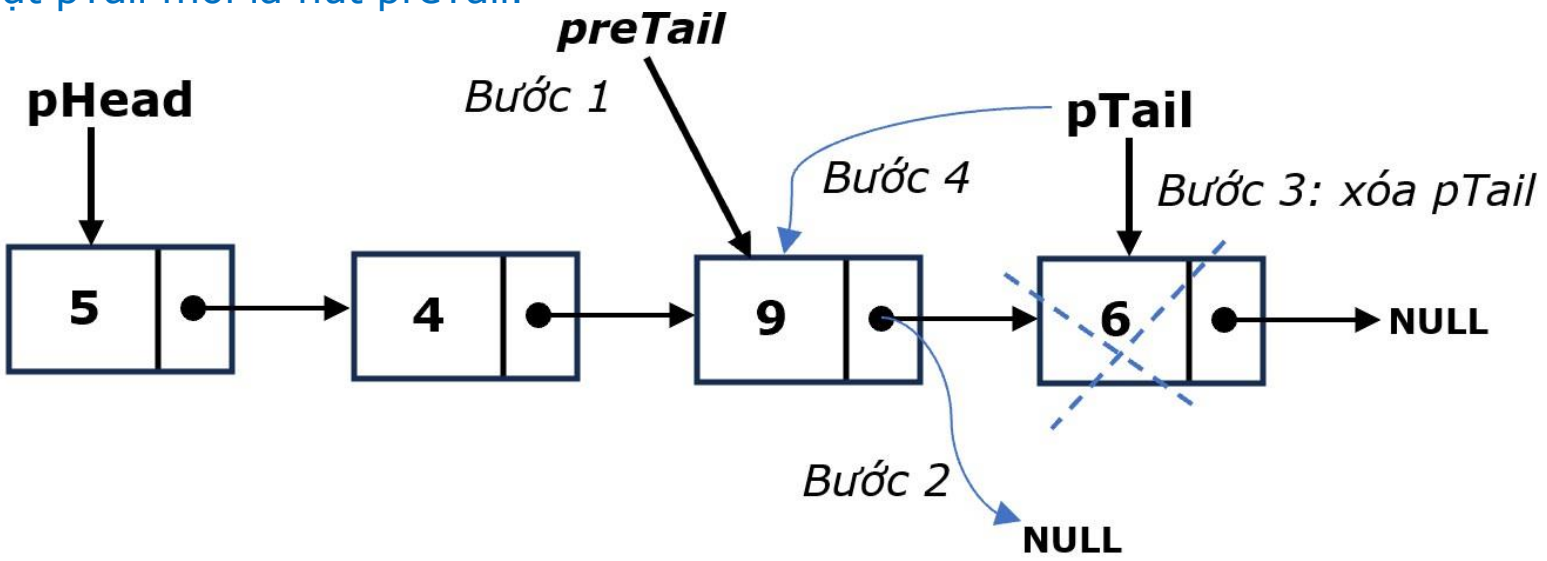
XÓA NÚT ĐẦU DANH SÁCH




```
void DeleteHead(LIST &l){
      NODE *p;
      if(IsEmpty(l)) return;
      p=l.pHead; //Bước 1
      l.pHead = l.pHead->pNext; //Bước 2
      free(p); //delete p; //Bước 3
      if(l.pHead==NULL) l.pTail=NULL; //Bước 4
}
```

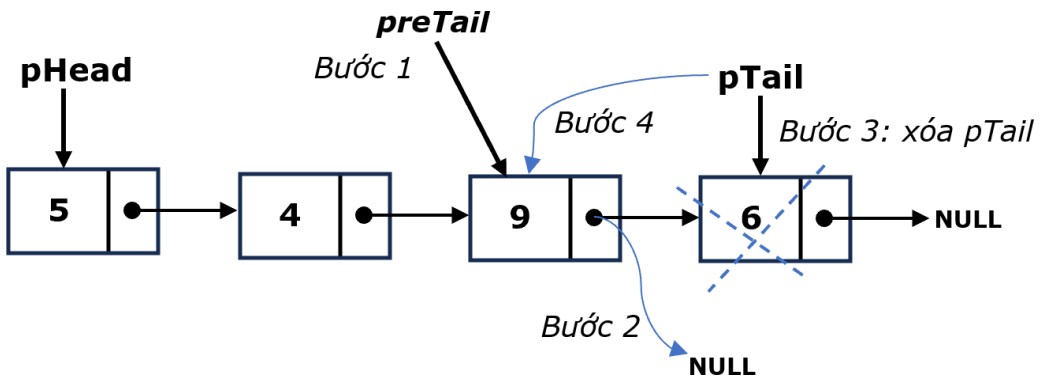
XÓA NÚT CUỐI DANH SÁCH

- **Trường hợp 1:** nếu danh sách rỗng, ta không làm gì cả.
- **Trường hợp 2:** nếu danh sách chỉ có một phần tử, ta hủy nút pHead, cập nhật cả pHead và pTail bằng NULL.
- **Trường hợp 3:** nếu danh sách có nhiều hơn một phần tử, ta thực hiện các bước sau:
  - Bước 1: duyệt danh sách để tìm nút ngay trước nút cuối, gọi là nút preTail.
  - Bước 2: cập nhật preTail->pNext bằng NULL.
  - Bước 3: hủy nút pTail.
  - Bước 4: cập nhật pTail mới là nút preTail.



XÓA NÚT CUỐI DANH SÁCH

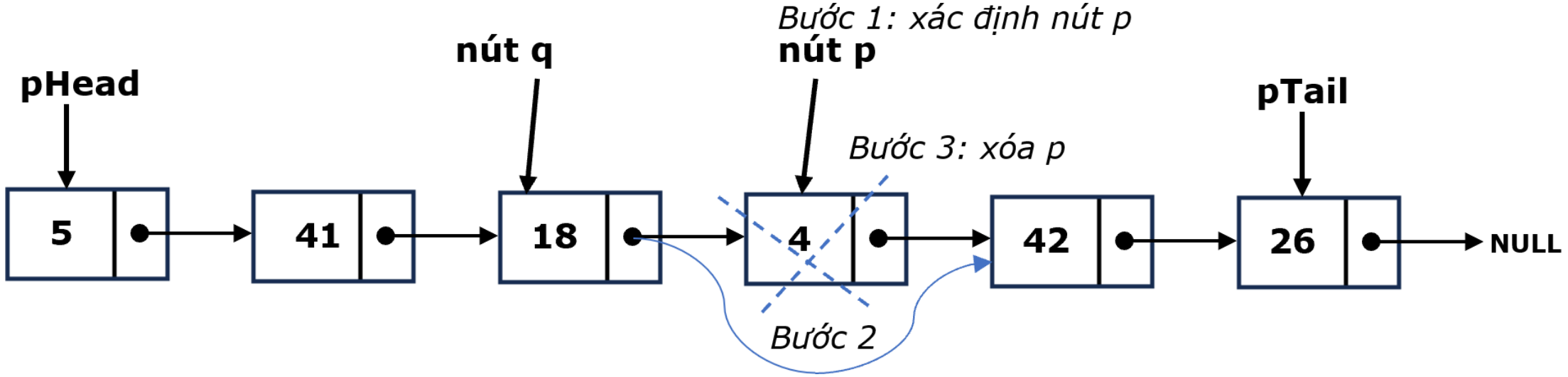
```
1   void DeleteTail(LIST &l) {
2        NODE* preTail;
3       if (l.pHead == NULL) return; //trường hợp 1
4       if (l.pHead == l.pTail) //trường hợp 2
5        {   free(l.pHead);
6           l.pHead = NULL;
7           l.pTail = NULL;
8           return;
9       }
10      //trường hợp 3
11      //Bước 1 - tìm nút trước pTail
12      preTail = l.pHead;
13      while (preTail->pNext != l.pTail) {
14          preTail = preTail->pNext;
15      }
16      preTail->pNext = NULL; //Bước 2
17      free(l.pTail); //Bước 3
18      l.pTail = preTail; //Bước 4
19  }
```



XÓA MỘT NÚT KHỎI DANH SÁCH

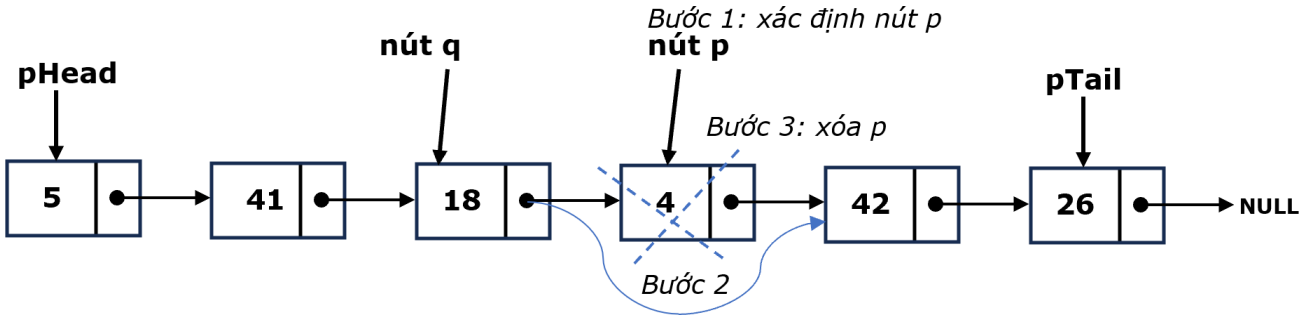
Cho một nút q trong danh sách liên kết đơn. Tác vụ sau đây sẽ xóa nút p nằm ngay sau nút q.

- Nếu nút q không tồn tại hoặc p không tồn tại, ta không làm gì cả.
- Nếu nút q tồn tại và p cũng tồn tại, ta thực hiện các bước sau đây.
  - Bước 1: xác định vị trí của p ngay sau q.
  - Bước 2: cho q->pNext trỏ đến nút ngay sau nút p.
  - Bước 3: hủy nút p.



XÓA MỘT NÚT KHỎI DANH SÁCH

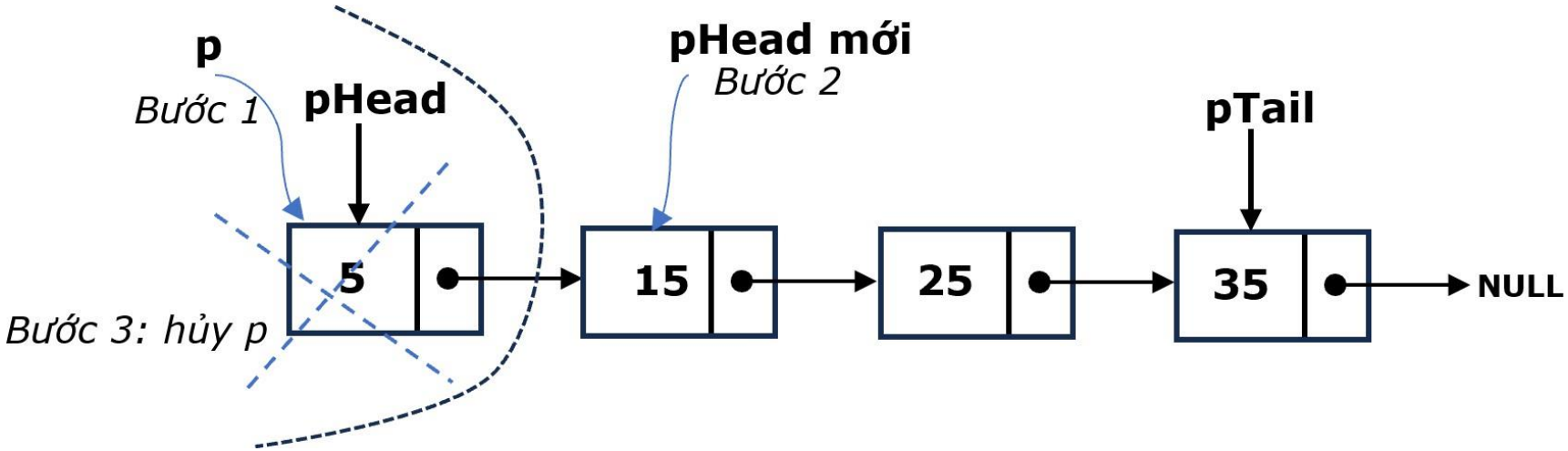
```
void DeleteAfter(LIST &l, NODE* q)
{
      NODE* p;
      //trường hợp q hoặc p không tồn tại
      if (q == NULL || q->pNext == NULL) return;
      //trường hợp q và p đều tồn tại
      p = q->pNext;  // Bước 1, p là nút ngay sau q
      q->pNext = p->pNext; // Bước 2
      if(p==l.pTail) l.pTail = q;
      free(p); //Bước 3
}
```



HỦY DANH SÁCH

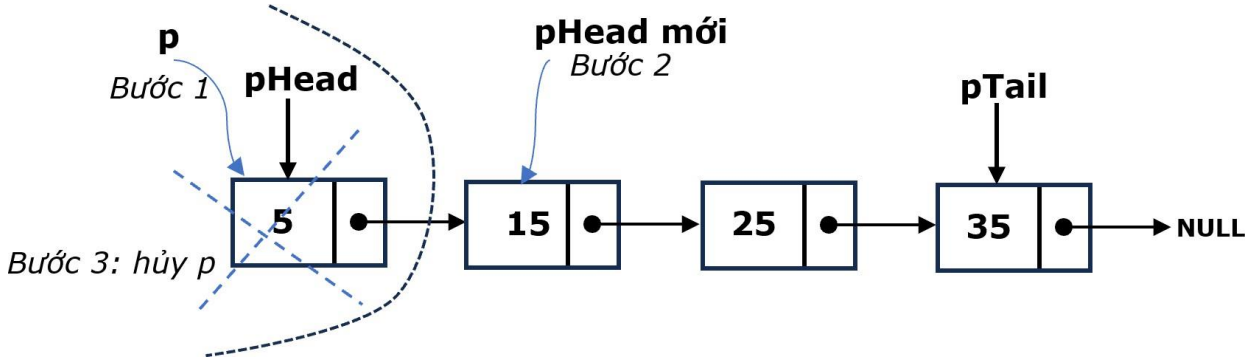
CÁC BƯỚC THỰC HIỆN:

- Khai báo một con trỏ: NODE* p;
- Khi nút pHead vẫn còn tồn tại thì:
  - Bước 1: cho p trỏ đến pHead.
  - Bước 2: cho pHead trỏ đến nút ngay sau nút p;
  - Bước 3: hủy nút p.
- Lặp lại các bước cho đến khi pHead không còn.
- Cập nhật lại pHead và pTail đều bằng NULL.



HỦY DANH SÁCH

```
1   void DeleteList(LIST &l)
2   {
3       NODE* p;
4       while(l.pHead!=NULL)
5       {
6           p = l.pHead; //Bước 1
7           l.pHead = p->next; //Bước 2
8           free(p); //Bước 3
9       }
10  }
```



SẮP XẾP DANH SÁCH

```
void InterchangeSort(int a[], int n)
{
        int i, j;
        for (i = 0 ; i<n-1 ; i++)
             for (j =i+1; j<n ; j++)
                  if(a[i] > a[j])
                  {
                       int tam=x;
                       x=y;
                       y=tam;
                  }
}
```

```
void InterchageSort(LIST &l)
{
     NODE *i, *j;
     for(i=l.pHead;i!=l.pTail;i=i->pNext)
          for(j=i->pNext;j!=NULL;j=j->pNext)
               if(i->key > j->key)
               {
                    int t = i->key;
                    i->key = j->key;
                    j->key = t;
               }
}
```

TÌM KIẾM MỘT PHẦN TỬ

```
int LinearSearch(int a[], int n, int x)
{
   int i;
   for(i=0; i<n ;i++)
         if(a[i]==x)
               return i;
   return -1;
}
```

```
NODE* LinearSearch(LIST l, int x)
{
   NODE *i;
   for(i=l.pHead; i!=NULL ;i=i->pNext)
         if(i->key == x)
               return i;
   return NULL;
}
```

XÁC ĐỊNH ĐỊA CHỈ VÀ VỊ TRÍ CỦA MỘT NÚT

XÁC ĐỊNH ĐỊA CHỈ CỦA MỘT NÚT

```
NODE* NodePointer(LIST l, int position) //position>=0
{
    int i=0;
    NODE* p;
    for(p=l.pHead;p!=NULL;p=p->pNext)
    {
        if(position==i)
            return p;
        i++;
    }
    return NULL;
}
```

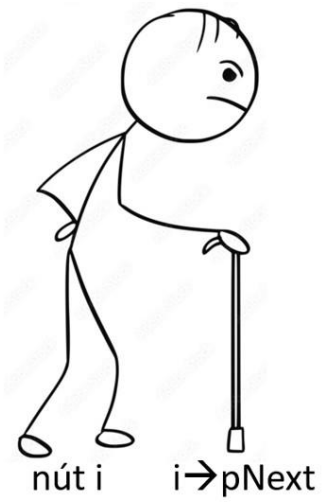
XÁC ĐỊNH VỊ TRÍ CỦA MỘT NÚT

```
int Position(LIST l, NODE* q)
{
    int i=0;
    NODE *p;
    for(p=l.pHead;p!=NULL;p=p->pNext)
    {
        if(p->key == q->key)
            return i;
        i++;
    }
    return -1;
}
```

XÁC ĐỊNH NÚT ĐỨNG TRƯỚC

- Cho trước một nút q, tác vụ này tìm nút đứng ngay trước nút q là nút nào. Tác vụ được viết thành hàm PreNode như sau:

```
NODE* PreNode(LIST l, NODE* q){
    NODE* i;
    if(q==l.pHead) return NULL;
    for(i=l.pHead->pNext;i!=l.pTail;i=i->pNext)
        if((i->pNext)->key == q->key)
            return i;
    return NULL;
}
```



KỸ THUẬT ÔNG GIÀ VÀ CÂY GẬY

Trong đoạn code trên, ta sử dụng kỹ thuật "ông lão và cây gậy". Trong đó, nút i đóng vai trò là ông lão, còn (i->pNext) đóng vai trò là cây gậy của ông lão. Khi cây gậy của ông lão chạm vào giá trị cần tìm ((i->pNext)->key == q->key) thì ông lão đang đứng tại nút ngay trước nút q.

THÊM MỘT NÚT VÀO DANH SÁCH TĂNG

- Giả sử ta có một danh sách liên kết đã được sắp xếp tăng. Ta tiến hành chèn một nút p (có giá trị là p->key) vào danh sách sao cho sau khi chèn thì danh sách cũng tăng. Tác vụ này có hai trường hợp đặc biệt:
  - Trường hợp 1: giá trị nút p nhỏ hơn hoặc bằng giá trị nút pHead, thì ta gắn nút p vào đầu danh sách.
  - Trường hợp 2: giá trị của nút p lớn hơn hoặc bằng giá trị nút pTail, thì ta gán nút p vào cuối danh sách.
- Nếu nút p không thuộc hai trường hợp trên thì ta sẽ dùng kỹ thuật "ông lão và cây gậy" để dò tìm vị trí thích hợp trong danh sách để chèn p vào.

THÊM MỘT NÚT VÀO DANH SÁCH TĂNG

```
void InsertIncrease(LIST &l, NODE* p)
{
    NODE* i;
    if(p->key <= l.pHead->key) //trường hợp 1
    {
        InsertHead(l,p);
        return;
    }

    if(p->key >= l.pTail->key) //trường hợp 2
    {
        InsertTail(l,p);
        return;
    }

    for(i=l.pHead;i!=l.pTail;i=i->pNext)
        if(p->key <= (i->pNext)->key)
        {
            InsertAfter(l,i,p);
            return;
        }
}
```

BÀI TẬP

BÀI 1: Tìm hiểu thật kỹ, viết code lại lần nữa cho các tác vụ trên danh sách liên kết đơn mà slide này đã trình bày. Rồi sau đó, tiến hành viết code cho các tác vụ sau đây:

1) Tính trung bình cộng giá trị của các nút trong danh sách liên kết đơn.

2) Đếm số lượng nút có giá trị chẵn.

3) Tìm nút có giá trị lớn nhất.

4) Tách danh sách list ban đầu thành hai danh sách list_chan và danh sách list_le sao cho list_chan chứa các giá trị chẵn và list le chưa các giá trị lẻ từ danh sách list ban đầu.

5) Cho hai danh sách liên kết đơn list1 và list2 đã sắp xếp tăng. Hãy ghép hai danh sách này lại thành danh sách list cũng sắp xếp tăng. Yêu cầu, lần lượt xét giá trị của từng cặp nút của hai danh sách, nút nào nhỏ hơn thì gắn vào danh sách mới. Không được ghép danh sách này vào cuối danh sách kia, rồi sau đó thực hiện giải thuật sắp xếp danh sách kết quả.

BÀI TẬP

BÀI 2 Hãy xây dựng một danh sách liên kết đơn mà phần dữ liệu của một nút là một struct sinh viên. Mỗi sinh viên có các thông tin: họ tên, mã sinh viên, điểm trung bình. Viết các tác vụ cơ bản cho danh sách liên kết này.

BÀI 3 Để biểu diễn các số lớn (Big Number hay còn gọi là Large Number) bằng danh sách liên kết đơn, ta có thể cho mỗi nút của danh sách lưu trữ một chữ số của số lớn đó. Với cách này, ta có thể biểu diễn và thực hiện các phép toán trên các số lớn mà không bị giới hạn bởi kích thước của kiểu dữ liệu nguyên thủy.

1) Xây dựng cấu trúc Node và List để biểu diễn Big Number:
   - Mỗi nút chứa một chữ số.
   - Danh sách liên kết để quản lý các nút.

2) Các tác vụ cơ bản:

   - Khởi tạo danh sách.
   - Thêm chữ số vào danh sách.
   - Hiển thị số lớn.
   - Thực hiện phép cộng, phép trừ hai số lớn.

BÀI TẬP

BÀI 4 Sử dụng danh sách liên kết đơn quản lý đa thức.

- Mục tiêu: sử dụng danh sách liên kết đơn để biểu diễn và thực hiện các thao tác cơ bản trên một đa thức.
- Mô tả: một đa thức trong toán học có dạng tổng quát:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$$

Các yêu cầu:

- Khai báo cấu trúc dữ liệu: Sử dụng danh sách liên kết đơn để biểu diễn một đa thức. Mỗi nút trong danh sách sẽ lưu trữ một hệ số và bậc của biến x.
- Các thao tác trên đa thức:
  - Khởi tạo đa thức: khởi tạo một đa thức rỗng.
  - Thêm một đơn thức (gồm hệ số và bậc) vào đa thức.
  - Hiển thị đa thức: in ra đa thức dưới dạng tổng các đơn thức.
  - Tính giá trị đa thức.

Q&A!